



PROJEKT SYSTEMU INFORMATYCZNEGO: STUDIO GIER WIDEO

| 1. Modelowanie Biznesowe

1.1. Opis organizacji

Organizacją jest **studio developerskie zajmujące się tworzeniem gier wideo**, którego działalność obejmuje projektowanie, produkcję, testowanie, publikację oraz dalszy rozwój gier komputerowych i mobilnych. Studio realizuje projekty samodzielnie lub we współpracy z partnerami zewnętrznymi, takimi jak wydawcy, platformy dystrybucyjne czy podwykonawcy odpowiedzialni za wybrane elementy produkcji.

Podstawowym celem organizacji jest tworzenie wysokiej jakości produktów cyfrowych, które spełniają oczekiwania graczy oraz wymagania rynku. Proces produkcyjny rozpoczyna się od opracowania koncepcji gry, przygotowania dokumentacji projektowej oraz zaplanowania harmonogramu prac. Następnie realizowane są zadania związane z programowaniem mechanik gry, tworzeniem grafiki 2D/3D, animacji, dźwięku oraz integracją wszystkich elementów w spójną całość.

W strukturze organizacyjnej studia występują różne role i zespoły specjalistyczne. Do najważniejszych należą: **Project Manager**, odpowiedzialny za zarządzanie projektem i koordynację zespołu, **Programiści**, tworzący kod źródłowy gry, **Graficy**, przygotowujący zasoby wizualne, **Game Designerzy**, projektujący mechaniki rozgrywki, oraz **Testerzy QA**, odpowiedzialni za kontrolę jakości produktu i zgłaszanie błędów.

W toku działalności studio korzysta z narzędzi informatycznych wspierających zarządzanie projektami, planowanie zadań, wersjonowanie kodu źródłowego, przechowywanie assetów, obsługę zgłoszeń błędów oraz monitorowanie postępu prac. Dzięki temu możliwe jest sprawne zarządzanie procesem produkcji oraz współpraca wielu osób nad jednym projektem.

Po zakończeniu etapu produkcji studio przygotowuje wersję końcową gry, publikuje ją na wybranych platformach dystrybucyjnych, prowadzi działania marketingowe oraz rozwija produkt poprzez aktualizacje, poprawki i dodatkową zawartość. Organizacja funkcjonuje zgodnie z obowiązującymi przepisami prawa, w szczególności dotyczącymi ochrony danych osobowych, prawa autorskiego oraz rozliczeń podatkowych.

1.2. Opis Kontekstu dziedziny problemowej

Dziedzina problemowa

Dziedziną problemową jest proces tworzenia **projektu gry wideo** w studiu developerskim. Proces ten obejmuje wszystkie etapy życia produktu – od powstania koncepcji gry, przez projektowanie i implementację, aż po testowanie, publikację oraz dalszy rozwój.

Tworzenie gry wymaga koordynacji pracy wielu specjalistów oraz zarządzania różnorodnymi zasobami, takimi jak kod źródłowy, zasoby graficzne (assets), modele 3D czy dźwięki. W trakcie realizacji projektu powstają również zadania oraz zgłoszenia błędów, które muszą być przypisywane odpowiednim członkom zespołu i monitorowane.

Zakres systemu

System informatyczny wspomaga proces tworzenia projektu gry wideo w studiu developerskim. Obejmuje on zarządzanie całym cyklem życia projektu – od planowania prac aż do publikacji gry.

System wspiera pracę zespołu projektowego oraz umożliwia efektywne zarządzanie zasobami, zadaniami i postępem projektu.

Funkcjonalności systemu

System umożliwia:

- planowanie i zarządzanie zadaniami w projekcie gry,
- zarządzanie zespołem projektowym oraz rolami użytkowników,
- przechowywanie i wersjonowanie zasobów (assetów),
- zarządzanie repozytorium kodu źródłowego,
- zgłaszanie i obsługę błędów w projekcie,
- monitorowanie postępu prac nad projektem,
- przygotowywanie buildów gry,
- publikację gry na platformach dystrybucyjnych.

Regulacje prawne

Działalność studia developerskiego oraz system informatyczny wspierający proces tworzenia gier muszą być zgodne z obowiązującymi przepisami prawa, w szczególności:

- Ustawą o prawie autorskim i prawach pokrewnych,
- Rozporządzeniem o ochronie danych osobowych (RODO),
- Ustawą Prawo przedsiębiorców,
- Ustawą o podatku od towarów i usług (VAT),
- Ustawą o świadczeniu usług drogą elektroniczną

1.2. Procesy i aktorzy biznesowi(Zewnętrzni)

Proces	Aktor biznesowy	Funkcje/zadania	Dane
Co się dzieje?	Kto uczestniczy spoza organizacji?	Jakie czynności są wykonywane?	Jakie dane?
Opracowanie pomysłu na grę	Wydawca gry	- Studio przedstawia koncepcję gry - Wydawca analizuje projekt i podejmuje decyzję o współpracy	opis projektu dokument koncepcji gry
Tworzenie assetów gry	Zewnętrzne studio graficzne	- Studio zamawia assety - Studio graficzne przygotowuje modele, tekstury lub animacje	modele 3D grafiki pliki assetów
Testowanie gry	Beta Testerzy	- Testerzy testują wersję gry - Zgłaszają błędy do zespołu developerskiego	raporty błędów dane testowe
Publikacja gry	Platforma dystrybucji (np. sklep z grami)	- Zespół przygotowuje wersję produkcyjną gry - Gra zostaje opublikowana na platformie dystrybucyjnej	build gry pliki instalacyjne
Sprzedaż gry	Klienci / gracze	- Gracz kupuje i pobiera grę	dane zakupu
Rozliczanie działalności	Urząd skarbowy	- Studio rozlicza przychody ze sprzedaży gry	dane podatkowe

1.3. Procesy i aktorzy(Wewnętrzni)

Proces	Aktor	Funkcje/zadania	Dane
Co się dzieje?	Kto uczestniczy w organizacji?	Jakie czynności są wykonywane?	Jakie dane?
Zarządzanie projektem gry	Project Manager	- planuje harmonogram prac - przydziela zadania członkom zespołu - monitoruje postęp projektu - kontroluje terminy realizacji	harmonogram projektu lista zadań status prac raporty postępu
Tworzenie mechanik gry	Game Designer	- projektuje zasady rozgrywki - opracowuje balans gry - przygotowuje dokumentację projektową - definiuje poziomy i systemy gry	dokumentacja game designu opis mechanik scenariusze gry
Implementacja gry	Programista	- tworzy kod źródłowy gry - rozwija funkcjonalności - naprawia błędy techniczne - integruje systemy gry	kod źródłowy repozytorium kodu zgłoszenia błędów
Tworzenie assetów	Grafik 2D/3D	- przygotowuje modele 3D - tworzy tekstury i animacje - projektuje interfejs użytkownika - aktualizuje zasoby wizualne	modele 3D grafiki 2D animacje pliki assetów
Testowanie gry	Tester QA	- testuje funkcjonalności gry - wykrywa błędy - raportuje problemy - weryfikuje poprawkę	raporty błędów wyniki testów scenariusze testowe
Administracja systemem	Administrator systemu	- zarządza kontami użytkowników - nadaje uprawnienia - konfiguruje system - monitoruje bezpieczeństwo	konta użytkowników role systemowe logi systemowe ustawienia systemu

Słownik

Studio developerskie – organizacja zajmująca się projektowaniem, tworzeniem oraz publikacją gier wideo.

Game Designer – osoba odpowiedzialna za projektowanie mechanik gry, zasad rozgrywki oraz fabuły.

Programista – osoba implementująca funkcjonalności gry w kodzie źródłowym przy użyciu silnika gry.

Grafik 2D/3D – osoba tworząca elementy wizualne gry, takie jak modele 3D, tekstury, animacje lub interfejs użytkownika.

Tester (QA) – osoba testująca grę w celu wykrycia błędów oraz sprawdzenia poprawności działania mechanik gry.

Project Manager – osoba zarządzająca projektem gry, planująca zadania oraz kontrolująca postęp prac zespołu.

Wydawca gry – firma finansująca lub wspierająca produkcję gry oraz odpowiadająca za jej dystrybucję.

Platforma dystrybucji gier – serwis umożliwiający publikację oraz sprzedaż gry użytkownikom (np. sklep z grami).

repozytorium kodu – miejsce przechowywania wersji kodu źródłowego projektu.

zadanie projektowe – jednostka pracy przypisana członkowi zespołu.

użytkownik systemu – osoba posiadająca konto w systemie.

Dane

projekty gier – nazwa projektu, opis, status realizacji, harmonogram, data rozpoczęcia, data zakończenia, przypisany zespół

użytkownicy systemu – imię, nazwisko, login, adres e-mail, hasło, rola, status konta, uprawnienia

zadania projektowe – tytuł zadania, opis, priorytet, termin realizacji, osoba przypisana, status wykonania

assety – nazwa pliku, typ assetu, wersja, autor, data dodania, lokalizacja pliku, powiązanie z projektem

repozytorium kodu – pliki źródłowe, commity, branche, autorzy zmian, historia wersji, znaczniki wydań

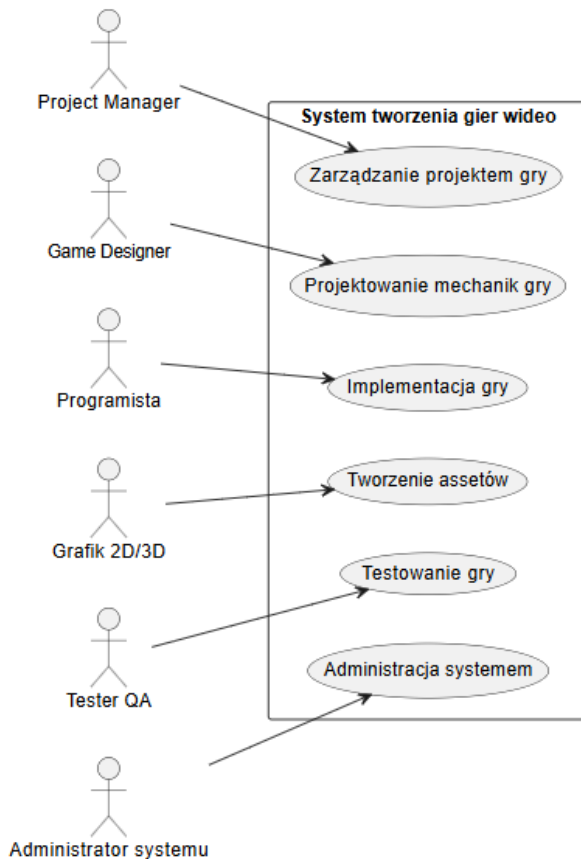
zgłoszenia błędów – tytuł błędu, opis, priorytet, status, osoba zgłaszająca, osoba odpowiedzialna, data zgłoszenia

buildy gry – numer wersji, data utworzenia, platforma docelowa, status publikacji, pliki instalacyjne

raporty postępu – procent realizacji projektu, wykonane zadania, opóźnienia, aktywność zespołu, terminy etapów

dane sprzedażowe – liczba sprzedanych kopii, przychody, platforma sprzedaży, region sprzedaży, data transakcji

1.4. Biznesowy diagram kontekstu systemu studia developerskiego



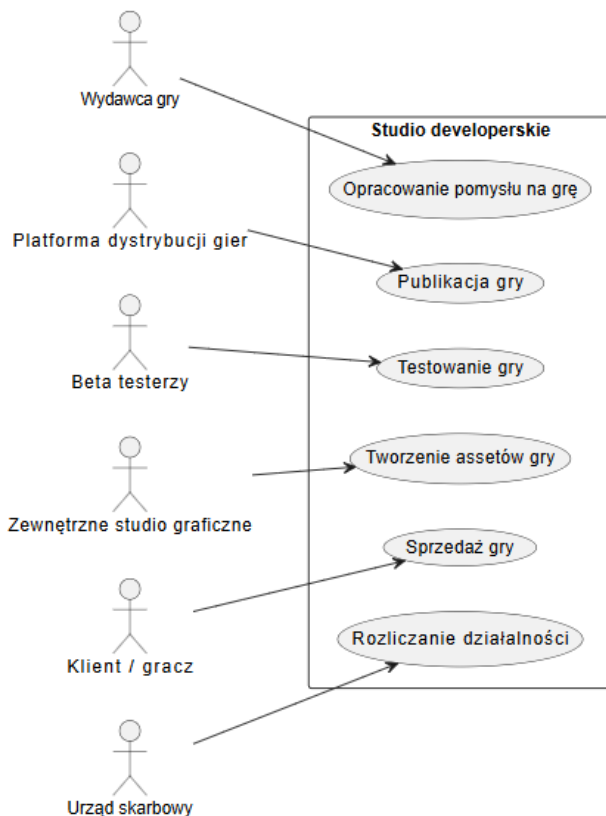
Aktorzy biznesowi

- Wydawca gry
- Platforma dystrybucji gier
- Beta testerzy
- Zewnętrzne studio graficzne
- Urząd Skarbowy

Procesy biznesowe

- Zgłoszenie pomysłu na grę
- Tworzenie assetów gry
- Testowanie gry
- Publikacja gry
- Sprzedaż gry
- Okresowe rozliczenie działalności

1.5. Wstępny diagram przypadków użycia systemu (aktorzy wewnętrzni)



Aktorzy wewnętrzni

- Project Manager
- Game Designer
- Programista
- Grafik 2D/3D
- Tester QA
- Administrator systemu

Procesy wewnętrzne

- Zarządzanie projektem gry
- Projektowanie mechanik gry
- Implementacja gry
- Tworzenie assetów
- Testowanie gry
- Administracja systemem

1.6. Cel i założenie projektu

Usprawnienie procesu tworzenia gry wideo w studio developerskim poprzez wsparcie zarządzania projektem gry, organizacji pracy zespołu, przechowywania assetów, obsługi zgłoszeń błędów oraz przygotowania i publikacji gry na platformach dystrybucyjnych.

| 2. Specyfikacja wymagań na SI – diagramy przypadków użycia

2.1. Cel i zadania SI - Systemowy słownik danych - Kontekstowy diagram przypadków użycia - Systemowy diagram przypadków użycia

Cel systemu

Celem systemu informatycznego jest wspomaganie procesu tworzenia gier wideo w studiu developerskim poprzez umożliwienie zarządzania projektami, organizacji pracy zespołu, kontroli postępu prac oraz obsługi zasobów projektowych.

System ma zwiększyć efektywność pracy zespołu, poprawić komunikację między członkami projektu oraz zapewnić centralne miejsce przechowywania danych związanych z projektem gry

Zadania systemu

System realizuje następujące zadania:

- umożliwia logowanie i autoryzację użytkowników
- zarządza kontami użytkowników oraz ich rolami
- umożliwia tworzenie i zarządzanie projektami gier
- wspiera planowanie oraz przydzielanie zadań projektowych
- umożliwia monitorowanie postępu prac nad projektem
- obsługuje repozytorium kodu źródłowego
- umożliwia przechowywanie i wersjonowanie assetów
- wspiera zgłaszanie oraz obsługę błędów
- umożliwia tworzenie buildów gry
- wspiera publikację gry na platformach dystrybucyjnych

2.2. Systemowy słownik danych

- **Użytkownik systemu** – osoba posiadająca konto w systemie, która może wykonywać operacje zgodnie z przypisaną rolą
- **Administrator** – użytkownik zarządzający systemem i kontami użytkowników
- **Projekt gry** – zbiór danych opisujących tworzoną grę (nazwa, harmonogram, status, zespół)
- **Zadanie projektowe** – jednostka pracy przypisana użytkownikowi w ramach projektu
- **Asset** – zasób cyfrowy wykorzystywany w grze (np. grafika, model 3D, dźwięk)
- **Repozytorium kodu** – system przechowujący kod źródłowy wraz z historią zmian
- **Zgłoszenie błędu** – zapis informacji o błędzie wykrytym w systemie lub grze
- **Build gry** – skompilowana wersja gry przeznaczona do testów lub publikacji
- **Publikacja gry** – proces udostępnienia gry na platformie dystrybucyjnej
- **Raport postępu** – zestawienie informacji o stanie realizacji projektu

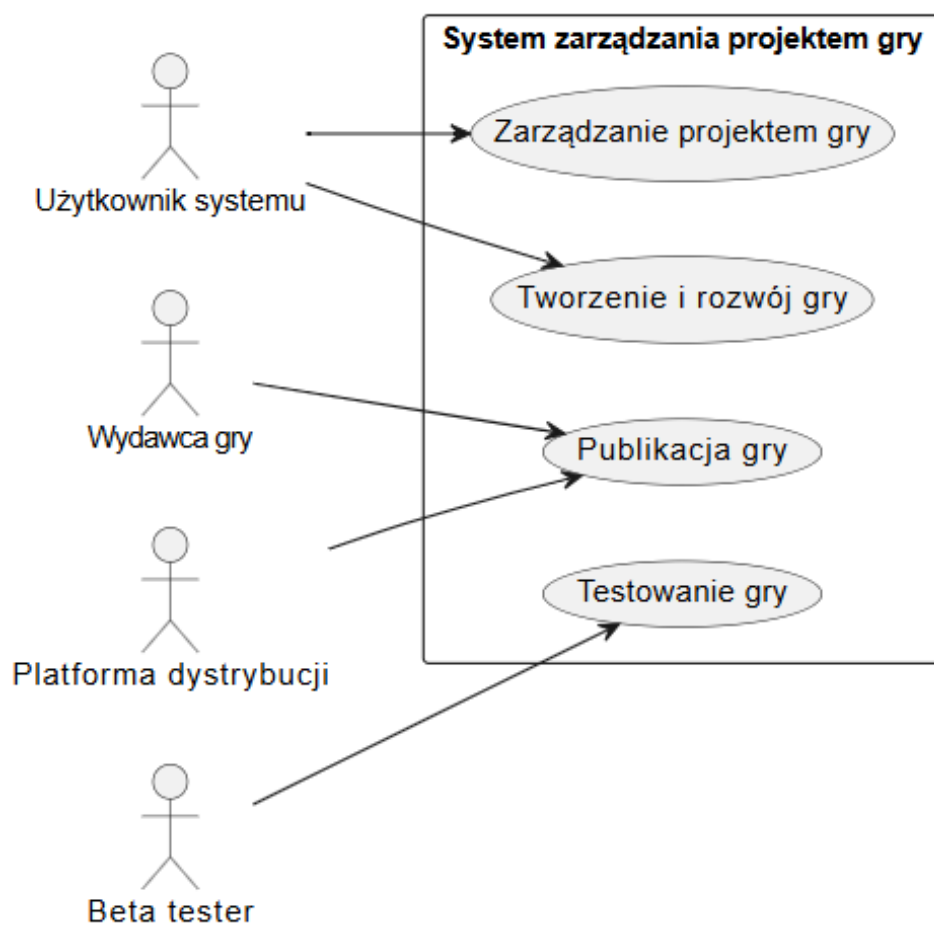
2.3. Kontekstowy diagram przypadków użycia

Opis

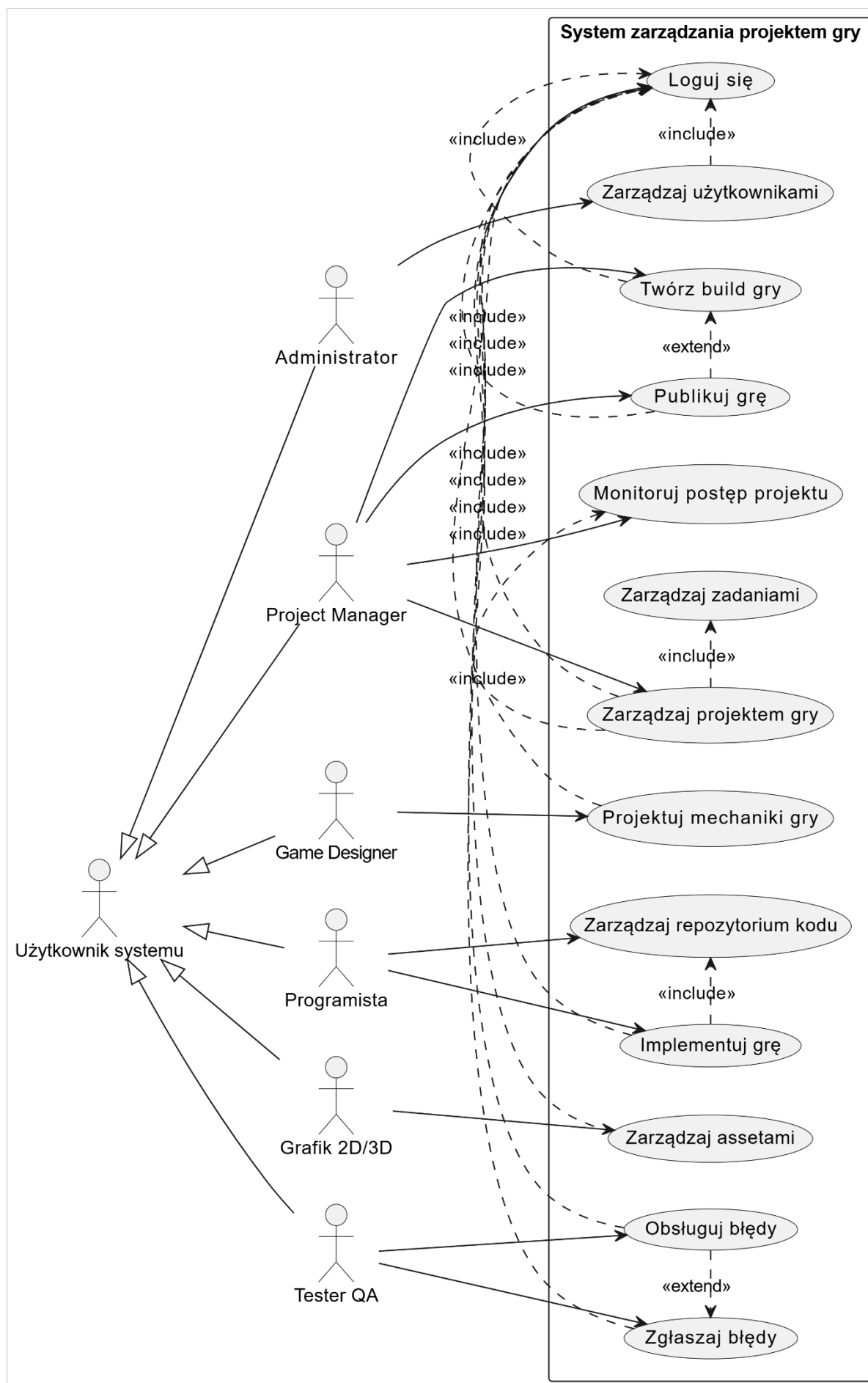
System: *System zarządzania projektem gry wideo*

Aktorzy:

- Użytkownik systemu
- Wydawca gry
- Platforma dystrybucji
- Beta tester



2.4. Systemowy diagram przypadków użycia



2.5. Scenariusze przypadków użycia

◆ S1 Logowanie użytkownika do systemu

S1.1. Opis

Funkcja umożliwia użytkownikowi zalogowanie się do systemu zarządzania projektem gry.

S1.2. Wspierane procedury i procesy biznesowe

Autoryzacja użytkowników systemu.

S1.3. Użytkownicy

- Administrator
- Project Manager
- Programista
- Grafik
- Tester QA
- Game Designer

S1.4. Warunki początkowe

Użytkownik posiada konto w systemie.

S1.5. Warunki końcowe

Użytkownik zostaje zalogowany i uzyskuje dostęp do systemu zgodnie ze swoją rolą.

S1.6. Przebieg główny

S1.6-1. Użytkownik uruchamia system.

S1.6-2. System wyświetla formularz logowania.

S1.6-3. Użytkownik wprowadza login i hasło.

S1.6-4. System weryfikuje dane.

S1.6-5. System przyznaje dostęp.

S1.6-6. System wyświetla panel główny.

S1.7. Przebiegi alternatywne

PA1.7.1-1 Wprowadzono niepoprawne dane logowania.

PA1.7.1-2 System wyświetla komunikat o błędzie.

S1.8. Sytuacje wyjątkowe

SW1.8.1-1 Brak połączenia z bazą danych.

Akcja: System wyświetla komunikat o błędzie i kończy operację.

S1.9. Reguły

- Każdy użytkownik musi posiadać unikalny login.
- Hasło musi spełniać wymagania bezpieczeństwa.

S1.10. Wymagania нефunkcjonalne

- System powinien umożliwić logowanie w czasie krótszym niż 2 sekundy.

S1.11. Uwagi i pytania otwarte

- Czy system ma obsługiwać odzyskiwanie hasła?

◆ S2 Zarządzanie projektem gry

S2.1. Opis

Funkcja umożliwia tworzenie, edycję oraz kontrolę projektów gier.

S2.2. Wspierane procedury i procesy biznesowe

Zarządzanie procesem produkcji gry.

S2.3. Użytkownicy

- Project Manager

S2.4. Warunki początkowe

Użytkownik jest zalogowany do systemu.

S2.5. Warunki końcowe

Projekt gry zostaje utworzony lub zaktualizowany.

S2.6. Przebieg główny

S2.6-1. Project Manager wybiera opcję zarządzania projektami.

S2.6-2. System wyświetla listę projektów.

S2.6-3. Użytkownik tworzy nowy projekt lub wybiera istniejący.

S2.6-4. Użytkownik wprowadza lub edytuje dane projektu.

S2.6-5. System zapisuje zmiany.

S2.7. Przebiegi alternatywne

PA2.7.1-1 Użytkownik nie wprowadził wymaganych danych.

PA2.7.1-2 System wyświetla komunikat o błędzie.

S2.8. Sytuacje wyjątkowe

SW2.8.1-1 Błąd zapisu danych.

Akcja: System informuje o błędzie i nie zapisuje zmian.

S2.9. Reguły

- Projekt musi mieć nazwę i datę rozpoczęcia.
- Każdy projekt musi mieć przypisanego managera.

S2.10. Wymagania niefunkcjonalne

- System powinien zapisywać dane w czasie rzeczywistym.

S2.11. Uwagi i pytania otwarte

- Czy możliwa jest archiwizacja projektów?
-

◆ S3 Zgłaszanie i obsługa błędów

S3.1. Opis

Funkcja umożliwia zgłaszanie oraz zarządzanie błędami w projekcie gry.

S3.2. Wspierane procedury i procesy biznesowe

Testowanie i kontrola jakości gry.

S3.3. Użytkownicy

- Tester QA
- Programista

S3.4. Warunki początkowe

Użytkownik jest zalogowany do systemu.

S3.5. Warunki końcowe

Błąd zostaje zapisany w systemie i przypisany do odpowiedniej osoby.

S3.6. Przebieg główny

S3.6-1. Tester wybiera opcję zgłoszenia błędu.

S3.6-2. System wyświetla formularz zgłoszenia.

S3.6-3. Tester wprowadza opis błędu.

S3.6-4. Tester zapisuje zgłoszenie.

S3.6-5. System zapisuje błąd i przypisuje go do programisty.

S3.7. Przebiegi alternatywne

PA3.7.1-1 Nie podano opisu błędu.

PA3.7.1-2 System odmawia zapisania zgłoszenia.

S3.8. Sytuacje wyjątkowe

SW3.8.1-1 System nie zapisuje zgłoszenia.

Akcja: Użytkownik próbuje ponownie.

S3.9. Reguły

- Każde zgłoszenie musi mieć priorytet.
- Każdy błąd musi być przypisany do osoby.

S3.10. Wymagania niefunkcjonalne

- System powinien umożliwiać zapis zgłoszenia w czasie krótszym niż 2 sekundy.

S3.11. Uwagi i pytania otwarte

- Czy system ma wysyłać powiadomienia o nowych błędach?

◆ S4 Tworzenie builda gry

S4.1. Opis

Funkcja umożliwia przygotowanie wersji gry (builda) do testów lub publikacji.

S4.2. Wspierane procedury i procesy biznesowe

Proces przygotowania wersji produkcyjnej gry.

S4.3. Użytkownicy

- Project Manager
- Programista

S4.4. Warunki początkowe

- Użytkownik jest zalogowany
- Projekt gry istnieje
- Kod źródłowy i assety są dostępne

S4.5. Warunki końcowe

Zostaje utworzony build gry gotowy do testów lub publikacji.

S4.6. Przebieg główny

S4.6-1. Użytkownik wybiera opcję tworzenia builda.

S4.6-2. System wyświetla konfigurację builda.

S4.6-3. Użytkownik wybiera platformę docelową.

S4.6-4. Użytkownik uruchamia proces budowania.

S4.6-5. System kompiluje projekt.

S4.6-6. System zapisuje build gry.

S4.7. Przebiegi alternatywne

PA4.7.1-1 Wybrano nieobsługiwaną platformę.

PA4.7.1-2 System wyświetla komunikat o błędzie.

S4.8. Sytuacje wyjątkowe

SW4.8.1-1 Błąd kompilacji.

Akcja: System przerywa proces i wyświetla log błędów.

S4.9. Reguły

- Build musi mieć unikalny numer wersji
- Build musi być powiązany z projektem

S4.10. Wymagania niefunkcjonalne

- Proces budowania powinien być monitorowany w czasie rzeczywistym

S4.11. Uwagi i pytania otwarte

- Czy system ma automatyzować buildy (CI/CD)?
-

◆ S5 Publikacja gry

S5.1. Opis

Funkcja umożliwia publikację gry na platformie dystrybucyjnej.

S5.2. Wspierane procedury i procesy biznesowe

Dystrybucja produktu cyfrowego.

S5.3. Użytkownicy

- Project Manager

S5.4. Warunki początkowe

- Użytkownik jest zalogowany
- Istnieje gotowy build gry

S5.5. Warunki końcowe

Gra zostaje opublikowana na platformie dystrybucyjnej.

S5.6. Przebieg główny

S5.6-1. Użytkownik wybiera opcję publikacji gry.

S5.6-2. System wyświetla listę buildów.

S5.6-3. Użytkownik wybiera build.

S5.6-4. Użytkownik wybiera platformę publikacji.

S5.6-5. System przesyła pliki gry.

S5.6-6. System potwierdza publikację.

S5.7. Przebiegi alternatywne

PA5.7.1-1 Brak dostępnego builda.

PA5.7.1-2 System blokuje publikację.

S5.8. Sytuacje wyjątkowe

SW5.8.1-1 Błąd połączenia z platformą.

Akcja: System przerywa operację i informuje użytkownika.

S5.9. Reguły

- Publikacja wymaga gotowego builda
- Publikacja musi być zatwierdzona przez Project Managera

S5.10. Wymagania нефункционалне

- System powinien zapewnić bezpieczne przesyłanie danych

S5.11. Uwagi i pytania otwarte

- Czy publikacja wymaga dodatkowej autoryzacji?
-

◆ **S6 Zarządzanie użytkownikami**

S6.1. Opis

Funkcja umożliwia zarządzanie kontami użytkowników w systemie.

S6.2. Wspierane procedury i procesy biznesowe

Administracja systemem.

S6.3. Użytkownicy

- Administrator

S6.4. Warunki początkowe

Administrator jest zalogowany do systemu.

S6.5. Warunki końcowe

Konto użytkownika zostaje utworzone, zmodyfikowane lub usunięte.

S6.6. Przebieg główny

S6.6-1. Administrator wybiera opcję zarządzania użytkownikami.

S6.6-2. System wyświetla listę użytkowników.

S6.6-3. Administrator wybiera operację (dodaj/edytuj/usuń).

S6.6-4. Administrator wprowadza dane.

S6.6-5. System zapisuje zmiany.

S6.7. Przebiegi alternatywne

PA6.7.1-1 Wprowadzono niepoprawne dane.

PA6.7.1-2 System wyświetla komunikat o błędzie.

S6.8. Sytuacje wyjątkowe

SW6.8.1-1 Brak uprawnień.

Akcja: System blokuje operację.

S6.9. Reguły

- Każdy użytkownik musi mieć przypisaną rolę
- Login musi być unikalny

S6.10. Wymagania нефункционалне

- System powinien zapewnić bezpieczeństwo danych użytkowników (RODO)

S6.11. Uwagi i pytania otwarte

- Czy użytkownicy mogą samodzielnie edytować swoje dane?

2.6. Dokumentacja przypadków użycia

◆ **PU1 – Logowanie do systemu**

Nazwa: Logowanie do systemu

Numer PU: PU1

Aktorzy:

- Użytkownik systemu

Krótki opis:

Umożliwia użytkownikowi dostęp do systemu po poprawnej autoryzacji.

Warunki wstępne:

- Użytkownik posiada konto w systemie

Warunki końcowe:

- Użytkownik jest zalogowany i ma dostęp do funkcji systemu

Główny przepływ zdarzeń:

- Użytkownik uruchamia system
- System wyświetla formularz logowania
- Użytkownik wprowadza login i hasło
- System weryfikuje dane
- System przyznaje dostęp

Alternatywne przepływy zdarzeń:

- Błędne dane logowania → system odmawia dostępu
- Konto zablokowane → system wyświetla komunikat

Specjalne wymagania:

- Szyfrowanie haseł
- Szybki czas logowania

Notatki:

- Możliwość dodania odzyskiwania hasła
-

◆ PU2 – Zarządzanie projektem gry

Nazwa: Zarządzanie projektem gry

Numer PU: PU2

Aktorzy:

- Project Manager

Krótki opis:

Umożliwia tworzenie i edycję projektów gier.

Warunki wstępne:

- Użytkownik jest zalogowany

Warunki końcowe:

- Projekt zostaje zapisany lub zmodyfikowany

Główny przepływ zdarzeń:

- Użytkownik wybiera moduł projektów
- System wyświetla listę projektów
- Użytkownik tworzy lub edytuje projekt
- System zapisuje dane

Alternatywne przepływy zdarzeń:

- Brak wymaganych danych → system odrzuca operację

Specjalne wymagania:

- Możliwość edycji w czasie rzeczywistym

Notatki:

- Możliwość archiwizacji projektów
-

◆ PU3 – Zgłaszanie błędów

Nazwa: Zgłaszanie błędów

Numer PU: PU3

Aktorzy:

- Tester QA

Krótki opis:

Umożliwia zgłaszanie błędów w projekcie gry.

Warunki wstępne:

- Użytkownik jest zalogowany

Warunki końcowe:

- Błąd zapisany w systemie

Główny przepływ zdarzeń:

- Tester otwiera formularz zgłoszenia

- Wprowadza dane błędu
- Zapisuje zgłoszenie
- System zapisuje dane

Alternatywne przepływy zdarzeń:

- Brak opisu → system blokuje zapis

Specjalne wymagania:

- Możliwość dodania załączników

Notatki:

- Możliwość przypisania błędu
-

◆ PU4 – Tworzenie builda gry

Nazwa: Tworzenie builda gry

Numer PU: PU4

Aktorzy:

- Programista
- Project Manager

Krótki opis:

Umożliwia generowanie wersji gry do testów lub publikacji.

Warunki wstępne:

- Projekt istnieje
- Użytkownik jest zalogowany

Warunki końcowe:

- Build gry został utworzony

Główny przepływ zdarzeń:

- Użytkownik wybiera opcję build
- Ustawia parametry
- Uruchamia proces
- System generuje build

Alternatywne przepływy zdarzeń:

- Błąd kompilacji → system przerywa proces

Specjalne wymagania:

- Monitorowanie procesu buildowania

Notatki:

- Możliwość automatyzacji
-

◆ PU5 – Publikacja gry

Nazwa: Publikacja gry

Numer PU: PU5

Aktorzy:

- Project Manager

Krótki opis:

Umożliwia publikację gry na platformie.

Warunki wstępne:

- Istnieje build gry
- Użytkownik zalogowany

Warunki końcowe:

- Gra opublikowana

Główny przepływ zdarzeń:

- Użytkownik wybiera publikację
- Wybiera build
- Wybiera platformę
- System publikuje grę

Alternatywne przepływy zdarzeń:

- Brak builda → brak możliwości publikacji

Specjalne wymagania:

- Bezpieczne przesyłanie danych

Notatki:

- Możliwe integracje z platformami
-

◆ PU6 – Zarządzanie użytkownikami

Nazwa: Zarządzanie użytkownikami

Numer PU: PU6

Aktorzy:

- Administrator

Krótki opis:

Umożliwia zarządzanie kontami użytkowników.

Warunki wstępne:

- Administrator zalogowany

Warunki końcowe:

- Konto użytkownika zmodyfikowane

Główny przepływ zdarzeń:

- Administrator wybiera moduł użytkowników
- Przegląda listę
- Dodaje/edytuje/usunie użytkownika
- System zapisuje zmiany

Alternatywne przepływy zdarzeń:

- Błędne dane → system odrzuca operację

Specjalne wymagania:

- Ochrona danych (RODO)

Notatki:

- Możliwość resetowania hasła

| 3. Modelowanie analityczne SI – model analityczny

3.1. Model analityczny SI

◆ MODEL ANALITYCZNY – OGÓLNY**◆ Encje (Entity)**

Na podstawie słownika danych:

- Użytkownik
- ProjektGry
- Zadanie
- Asset
- RepozytoriumKodu
- ZgłoszenieBłędu
- BuildGry
- RaportPostępu

◆ Boundary (interfejsy)

- FormularzLogowania
- PanelGłówny
- FormularzProjektu
- FormularzZadania
- FormularzZgłoszeniaBłędu
- PanelBuildów

- PanelPublikacji
 - PanelAdministracyjny
-

♦ **Control (logika przypadków użycia)**

- LogowanieController
 - ProjektController
 - ZadanieController
 - BłądController
 - BuildController
 - PublikacjaController
 - UżytkownikController
-

SZCZEGÓŁOWE MODELE (NA PRZYKŁADACH PU)

♦ **PU1 – Logowanie do systemu**

Boundary:

- FormularzLogowania

Control:

- LogowanieController

Entity:

- Użytkownik

Relacje:

- Użytkownik → FormularzLogowania (wprowadza dane)
 - FormularzLogowania → LogowanieController
 - LogowanieController → Użytkownik (weryfikacja)
-

♦ **PU2 – Zarządzanie projektem**

Boundary:

- FormularzProjektu
- PanelProjektów

Control:

- ProjektController

Entity:

- ProjektGry
- Użytkownik

Relacje:

- ProjektController zarządza ProjektGry
 - Użytkownik tworzy/edytuje projekt
-

♦ **PU3 – Zgłaszanie błędów**

Boundary:

- FormularzZgłoszeniaBłędu

Control:

- BłądController

Entity:

- ZgłoszenieBłędu
- Użytkownik

Relacje:

- Tester → Formularz
 - Formularz → Controller
 - Controller zapisuje Zgłoszenie
-

♦ **PU4 – Tworzenie builda**

Boundary:

- PanelBuildów

Control:

- BuildController

Entity:

- BuildGry
- ProjektGry
- RepozytoriumKodu
- Asset

Relacje:

- BuildController pobiera dane z repozytorium i assetów
 - Tworzy BuildGry
-

◆ PU5 – Publikacja gry**Boundary:**

- PanelPublikacji

Control:

- PublikacjaController

Entity:

- BuildGry
- ProjektGry

Relacje:

- Controller wysyła Build na platformę
-

◆ PU6 – Zarządzanie użytkownikami**Boundary:**

- PanelAdministracyjny

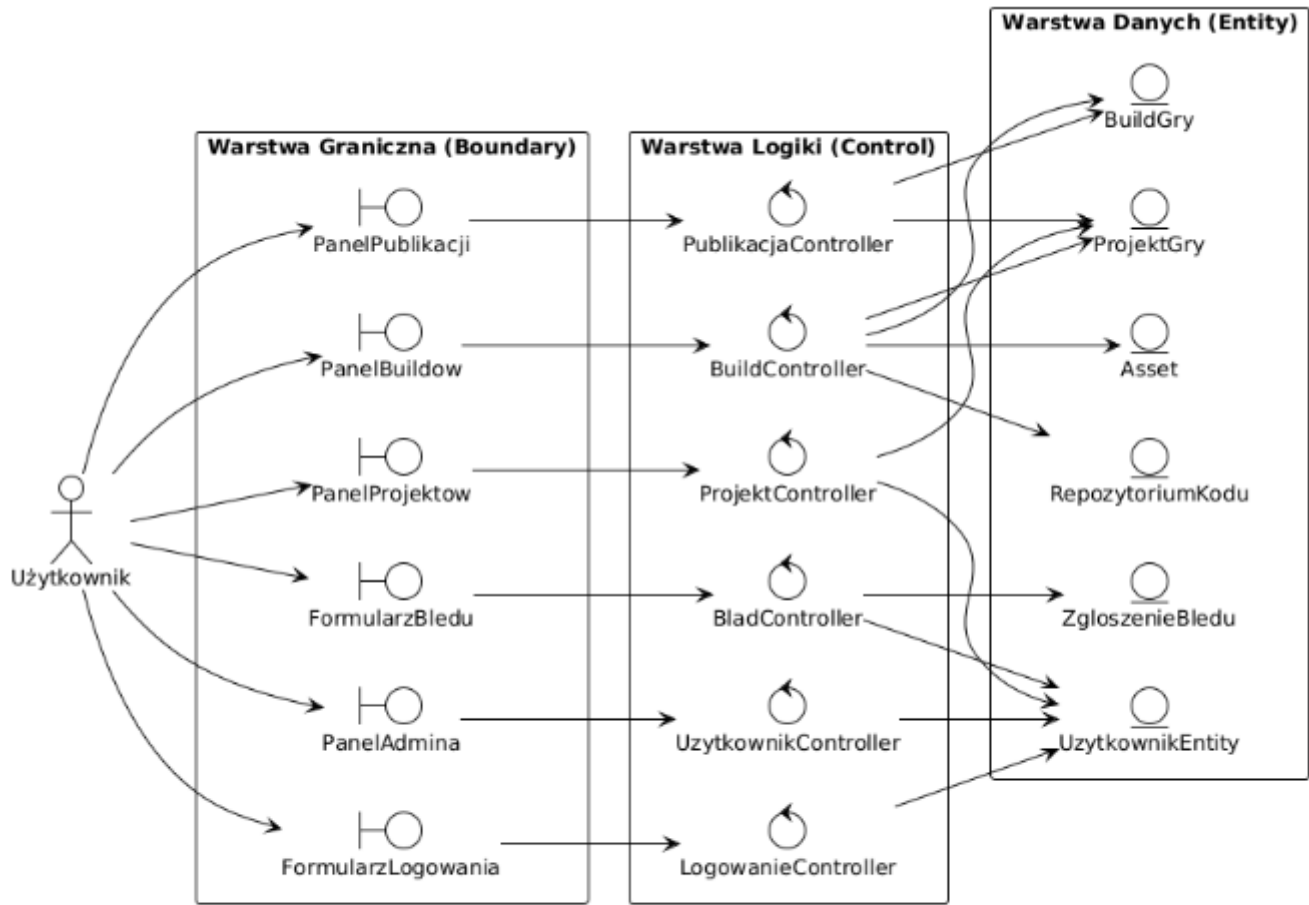
Control:

- UżytkownikController

Entity:

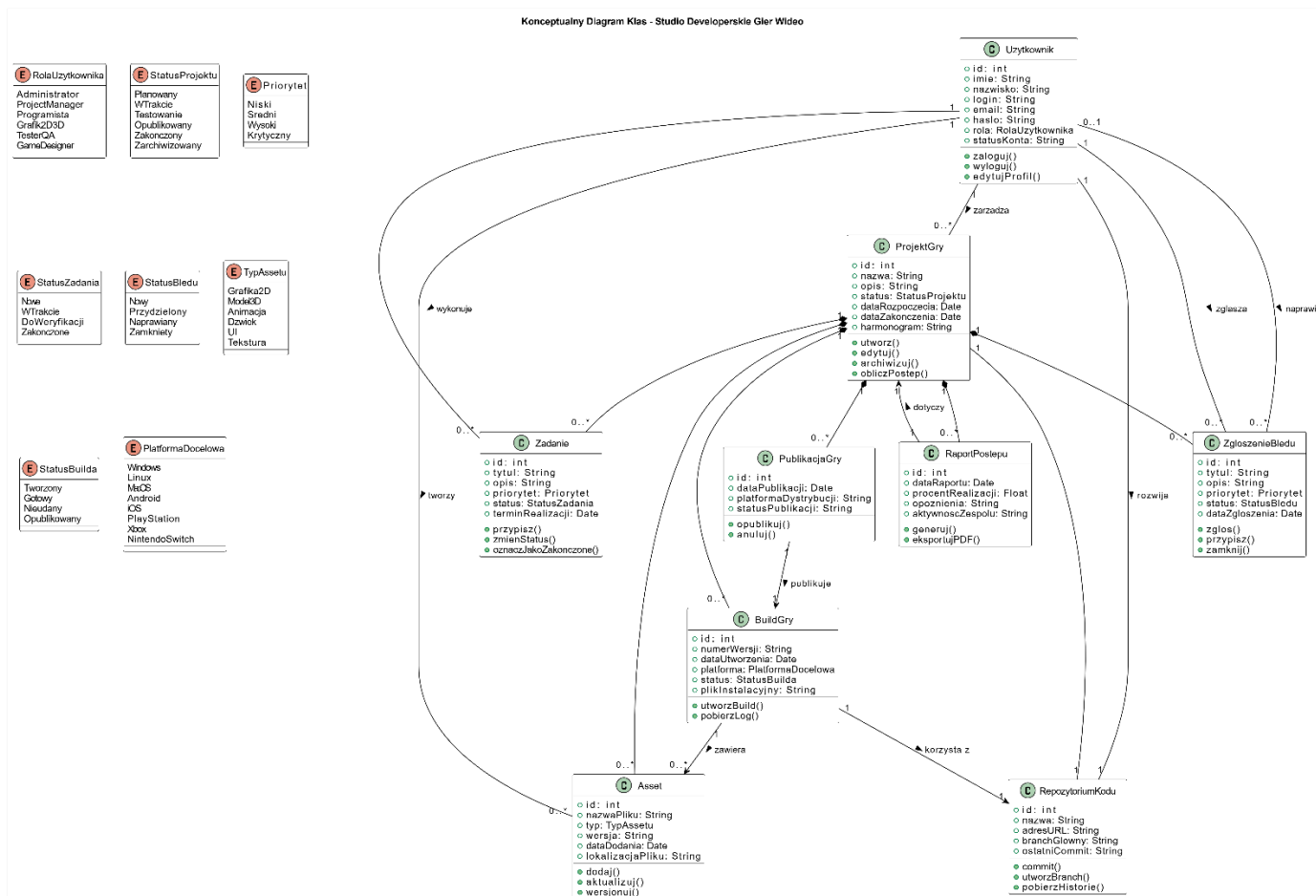
Użytkownik

3.2.Diagram modelu Analitycznego

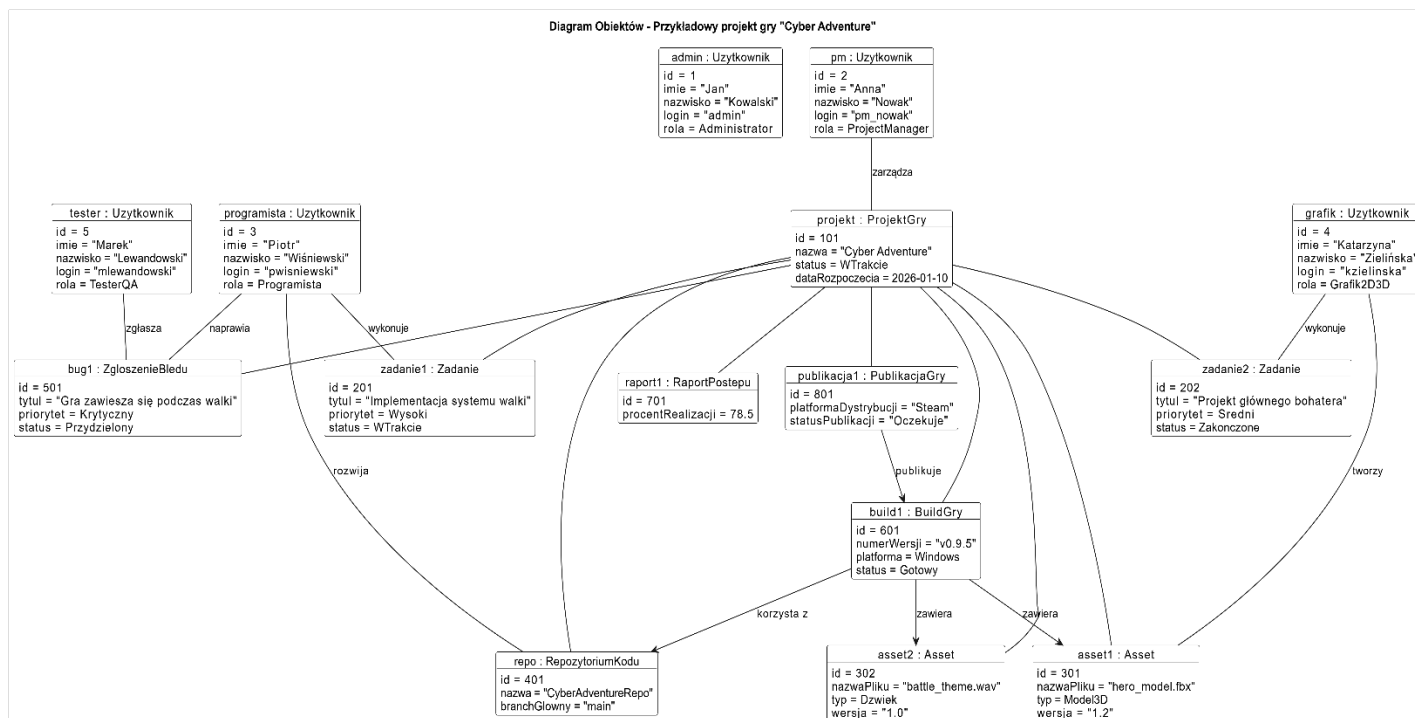


4. Modelowanie danych – diagram klas, diagram obiektów

4.1 Konceptualny diagram klas

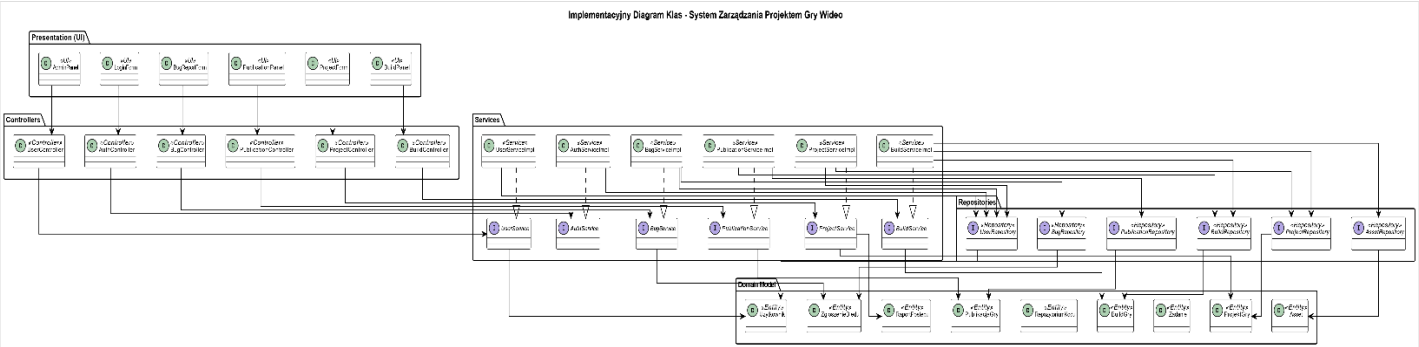


4.2 Diagram Obiektów



|5. Projektowanie danych

5.1. Implementacyjny diagram klas



5.2. Projekt relacyjnej bazy danych

